Local State-Space versus Global Learned Models for Probabilistic SKU-Level Demand Forecasting: A Rolling-Origin Study on the M5 Retail Dataset

Sayan Bhattacharya 2401020059
Aryu Rao 2401010107
Debasish Karan 2401010141
Saswataduity Bhuin 2401010425
Aryan Vibhuti 2401010105

Department of Computer Science and Artificial Intelligence (CSAI)
Newton School of Technology, Rishihood University, India
Applied Machine Learning Project — Phase 1 Report

***Abstract*—Retail replenishment depends on daily, item-level demand forecasts that are both accurate and honest about their uncertainty. This report asks whether a *local* model, fitted independently to each stock-keeping unit (SKU), or a *global* model, fitted jointly to all SKUs, better serves that need, and whether simple baselines remain competitive. Using a 40-SKU subset of the M5 Walmart dataset (store CA_1, category FOODS, 1,941 days), we compare six forecasters under an identical four-origin rolling backtest with a 28-day horizon: two seasonal-naive baselines, a pooled histogram gradient-boosting regressor, a per-SKU Kalman filter with a local linear trend (with and without weekly seasonality), and a global quantile LSTM. The global LSTM is the most accurate model on every view we examined: it attains a WAPE of 0.499 against 0.566 for gradient boosting and 0.623 for seasonal naive, wins at all four origins, in all four Syntetos–Boylan demand classes and in every weekly horizon block, and its 80% P10–P90 interval covers 81.2% of outcomes. The ranking is stable across three random seeds (WAPE 0.501 ± 0.003) and survives removal of the single SKU that accounts for 42.9% of test volume. The Kalman filter never wins: its intervals are valid but almost twice as wide as the LSTM's (coverage 95%, mean width 10.6 versus 6.0 units), and trend extrapolation inflates its error with the horizon. Gradient boosting is the strongest classical learner and the most cost-effective alternative. All features and models pass explicit, automated leakage tests.**

***Index Terms*—demand forecasting, intermittent demand, Kalman filter, state-space models, gradient boosting, LSTM, quantile regression, probabilistic forecasting, rolling-origin evaluation, M5 competition**

I. Introduction

INVENTORY decisions are made one item at a time. A retailer deciding how many units of a product to hold next month needs a forecast for that product, at a daily resolution, together with a credible range: a point forecast alone cannot set a safety stock. Two properties of retail data make this difficult. First, most item-level series are short on signal: in the full M5 dataset 68.0% of all daily sales cells are zero (Section III-B), so many SKUs sell sporadically. Second, items differ widely in volume and behaviour, so a method that suits a fast-moving staple may fail on a slow, lumpy specialty product.

Two families of methods respond to these difficulties in opposite ways. *Local* methods, such as exponential smoothing and the Kalman filter [1], [2], fit a separate model to each series and adapt quickly to that series' level. *Global* methods, such as pooled gradient boosting [3] and recurrent networks like DeepAR [4], fit one model across all series and share statistical strength between them. The M5 competition found that global, machine-learning approaches dominated the leaderboard [5], and Montero-Manso and Hyndman showed formally that global models need not be less expressive than local ones [6]. Neither result, however, tells a practitioner *which items* benefit, at *which horizons*, or whether the uncertainty estimates are trustworthy.

This study addresses three research questions under a single, controlled evaluation protocol:

RQ1 Does a global model forecast daily SKU demand more accurately than per-SKU local state-space models and simple baselines?

RQ2 Under which conditions (demand class, horizon, forecast origin) does a local Kalman filter remain competitive?

RQ3 How well calibrated are the probabilistic forecasts (P10/P50/P90) produced by each family?

The contributions of this Phase 1 work are as follows. (i) A documented, assertion-checked cleaning pipeline that turns the raw, wide-format M5 files into a long panel with an explicit, conservative availability proxy. (ii) A leakage-safe feature set whose safety is verified by two automated tests rather than asserted. (iii) A like-for-like comparison of six forecasters on identical SKUs, origins and horizons, reporting point accuracy, quantile loss and interval coverage. (iv) A disaggregated error analysis by origin, horizon, demand class and SKU, including a robustness check against volume concentration. (v) A public, reproducible code base.

The remainder of the report is organised as follows. Section II reviews related work. Sections III and IV describe the data and the exploratory analysis that motivated the design.

Section V defines the features, Section VI the models, and Section VII the evaluation protocol. Section VIII presents results, Section IX discusses them and the alternatives we did not adopt, and Section X sets out limitations and a measurable plan for Phase 2.

II. Literature Review

A. State-Space Models and the Kalman Filter

Kalman's recursive estimator [1] computes the minimum mean-squared-error estimate of a hidden linear state from noisy observations, and is optimal when the system is linear and the noise Gaussian. Harvey's structural time-series framework [2] casts forecasting components (level, slope, seasonality) as such states, so that the local linear trend model used here is a special case. Durbin and Koopman [7] give the modern treatment, including maximum-likelihood estimation of the noise variances, which this study replaces with a heuristic (Section VI-D). The attraction for inventory problems is that the filter adapts to level shifts without retraining and produces a forecast variance as a by-product.

B. Intermittent Demand

Croston [8] observed that exponential smoothing is biased on series with many zeros and proposed forecasting demand size and inter-demand interval separately. Syntetos and Boylan [9] quantified and corrected that bias, and Syntetos, Boylan and Croston [10] proposed the widely used classification of demand by the average inter-demand interval (ADI) and the squared coefficient of variation of non-zero demand sizes (CV^2), with cut-offs of 1.32 and 0.49. We use that classification to disaggregate errors, because a single pooled score hides class-specific failure.

C. Machine-Learning and Global Models

Gradient boosting [3] builds an additive ensemble of shallow trees, each fitted to the negative gradient of the loss; histogram-based implementations such as LightGBM [11] made it fast enough to pool millions of rows, and LightGBM variants were prominent among the top M5 entries [5]. Recurrent networks based on the LSTM cell [12] model sequences directly. DeepAR [4] trains a single autoregressive LSTM across many related series, with item embeddings and known-in-advance covariates, and outputs a parametric predictive distribution. MQ-RNN [13] instead emits multiple quantiles for all horizons at once, avoiding error accumulation from recursion; the Temporal Fusion Transformer [14] extends this direct multi-horizon design with attention. Our global LSTM follows the MQ-RNN pattern. Montero-Manso and Hyndman [6] supply the theoretical frame for the local-versus-global question.

D. Probabilistic Forecasting and Quantile Loss

Koenker and Bassett's regression quantiles [15] established that minimising the asymmetric pinball loss yields a conditional quantile. Gneiting and Raftery [16] showed that such losses are proper scoring rules, so they reward honest uncertainty. Independently trained quantiles can cross; Chernozhukov et al. [17] proved that monotone rearrangement (sorting) fixes crossing without degrading the estimate, which is the procedure we apply. For retail count data, Kolassa [18] argues that full predictive distributions, not point forecasts, should be evaluated.

TABLE I
Literature used and its role in this study

Strand	Refs.	Use in this study
State-space filtering	[1], [2], [7]	Local linear trend and seasonal Kalman models
Intermittent demand	[8]–[10]	ADI/CV^2 classes for error analysis
Gradient boosting	[3], [11]	Pooled tabular baseline
Global deep models	[4], [12]–[14]	Global quantile LSTM design
Local vs. global	[5], [6]	Framing of RQ1
Quantiles & scoring	[15]–[18]	Pinball loss, rearrangement, calibration
Evaluation	[19], [21]–[25]	Rolling origins, WAPE, leakage tests

E. Evaluation Methodology

Tashman [19] and Hyndman and Athanasopoulos [20] recommend rolling-origin evaluation over a single split, and Bergmeir and Benítez [21] discuss its validity for time series. Hyndman and Koehler [22] show that percentage errors break down when actuals are zero, which motivates the volume-weighted WAPE used here. Kolassa [23] shows that the optimal point forecast depends on the error measure (the median minimises MAE, the mean minimises MSE), a point that matters when comparing a quantile model against mean-forecasting models (Section IX-C). Ben Taieb et al. [24] compare recursive and direct multi-step strategies. Kaufman et al. [25] formalise data leakage, which we test for explicitly.

F. Research Gap

Published comparisons typically report one pooled score per method on a large benchmark. Few studies, and in particular few reproducible student-scale studies, place a local state-space model and a global learned model under *identical* rolling backtests and then examine *where* each wins: by demand class, by horizon, by origin and in the calibration of its intervals. That disaggregated view is the focus here. Table I summarises how each strand of the literature is used.

III. Dataset and Data Preparation

A. The M5 Dataset

The M5 Forecasting–Accuracy dataset [5] contains daily unit sales of 3,049 Walmart products in ten stores across California, Texas and Wisconsin. It consists of three files: `sales_train_evaluation.csv` (30,490 item-store series $\times$ 1,941 days, 29 January 2011 to 22 May 2016), `calendar.csv` (1,969 rows with dates, holidays and events, and SNAP food-assistance days per state) and `sell_prices.csv` (6,841,121 weekly store-item prices). The files were obtained from a

TABLE II
RAW-DATA AUDIT OF THE FULL M5 FILES AND TREATMENT ADOPTED

Issue	Extent	Treatment
Wide format, `d_1...d_1941`	1,941 cols	Melt to long panel; join calendar dates
Event fields empty	1,807/1,969 days	NaN → `"None"` (means no event)
Zero-sales cells	68.0%	Kept; never imputed
First sale after day 200	48.5% of series	Restrict subset to early starters
Item-store-weeks with price	79.6%	No imputation; `price_missing` flag
Duplicates / negatives	0 / 0	Asserted in pipeline
Size	30,490 series	One store, one category, 40 SKUs

Source: `notebooks/01_data_cleaning_eda.ipynb`, *Sec. 2.*

public Hugging Face mirror of the Kaggle release; their shapes match the official competition files exactly.

B. Raw Data Quality Audit

Before any cleaning, we measured what is wrong with the raw data. Table II lists each issue, its extent and the treatment adopted. Two findings shape everything downstream. Zeros are the norm, not the exception, and nearly half of all series start selling late, so leading zeros represent "not yet launched" rather than "no demand". Price records exist for only 79.6% of possible item-store-weeks, and a missing price indicates that the item was not on sale that week.

C. Subset Selection

The study uses one store (CA_1) and one category (FOODS). FOODS was chosen because its items are SNAP-eligible, which makes the SNAP calendar feature meaningful. Of the 1,437 CA_1 FOODS series, we retained those whose first sale occurs within the first 200 days (so every SKU has a long history) and whose mean daily sales are at least 0.3 units (so the series is not almost entirely zero). We then sorted the survivors by mean sales and took 40 evenly spaced SKUs, which yields a deliberate mix of fast and slow sellers rather than a random draw dominated by slow items. The resulting subset contains 23 FOODS_3, 11 FOODS_2 and 6 FOODS_1 items; their mean daily sales over the year before the first origin range from 0.11 to 47.95 units, with a median of 1.07.

D. Cleaning Pipeline

The function `build_panel` in `src/data.py` performs four steps: (1) melt the wide table into one row per SKU-day; (2) join the calendar, fill empty event fields with `"None"`, derive `is_event`, and select the SNAP flag of the store's state; (3) left-join weekly prices, preserving missing prices; (4) derive flags. A validation function, `check_panel`, then asserts that there are no duplicate SKU-dates, no negative sales, that every SKU has all 1,941 days, and that the date index has no gaps. The clean panel has 77,640 rows (40 SKUs $\times$ 1,941 days) and 24 columns and is stored as `data/processed/m5_clean_panel.csv`; a reload check confirms the saved and in-memory sales are identical.

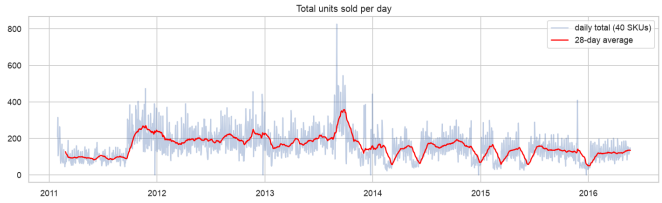


Fig. 1. Total daily unit sales of the 40 SKUs with a 28-day moving average. Source: notebook 01, Sec. 9.

E. Availability Proxy

M5 contains no stockout indicator, yet a zero may mean either "no one wanted the item" or "the item could not be bought". We define a deliberately conservative proxy,

$$\text{potential_unavailable}_{i,t} = \mathbb{I}[\, y_{i,t} = 0 \;\wedge\; \text{price}_{i,w(t)} \text{ missing}\,], \tag{1}$$

where $w(t)$ is the Walmart week containing day t. Of 33,327 zero-sales days in the panel (42.9%), 679 satisfy the proxy. Inspection shows that all 679 fall *before* the relevant SKU's first sale (pre-launch), none falls mid-history, and none falls inside a forecast window. We therefore report the proxy as a diagnostic only, never replace zeros with estimated demand, and state plainly that in this subset it identifies launches rather than stockouts.

F. Outlier Policy

Days with sales more than five standard deviations above the SKU's mean are flagged (`high_sales_flag`, 165 days, 0.21%) but neither removed nor capped. The largest, 599 units of FOODS_3_090 on 1 September 2013, is part of a multi-week surge rather than an isolated error. Removing genuine spikes would make every model look better than it would perform in deployment.

IV. EXPLORATORY DATA ANALYSIS

Each analysis below ends in a design decision, summarised in Table III.

Target distribution. Daily sales per SKU have mean 3.90, standard deviation 14.25, median 1 and maximum 599 units; 42.9% of SKU-days are zero. The target is a sparse, right-skewed count, so Gaussian assumptions are approximations and forecasts must be clipped at zero.

Level and trend. Total daily sales across the 40 SKUs (Fig. 1) shift upward in late 2011 as more SKUs launch, spike in August–September 2013 (driven by FOODS_3_090), drift thereafter, and drop to near zero each Christmas when the store closes. A non-stationary level motivates testing a filter that tracks a moving level.

Seasonality. Weekend sales exceed Monday–Thursday sales by 53% on average (Fig. 2); the monthly pattern is weaker. The autocorrelation of total sales peaks at lags 7, 14, 21 and 28 (Fig. 3). Weekly structure is the dominant signal, which justifies lags at multiples of seven, a day-of-week feature, the lag-7 seasonal-naive baseline and the seasonal extension of the Kalman filter.

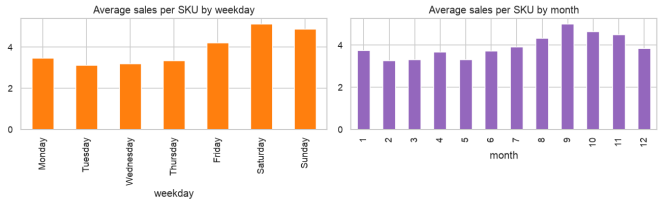


Fig. 2. Average sales per SKU by weekday (left) and by month (right). Source: notebook 01, Sec. 10.

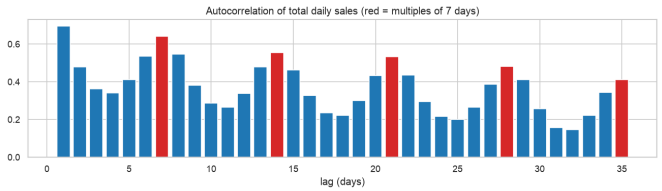


Fig. 3. Autocorrelation of total daily sales; multiples of seven days in red. Source: notebook 01, Sec. 10.

Events, SNAP and price. Mean sales are 4.155 units on SNAP days against 3.779 otherwise, an uplift of about 10%. Event days show no average uplift (3.841 versus 3.909), because events of different types push sales in opposite directions. The median correlation between weekly price and weekly sales across SKUs is -0.15: weak, and food prices move little. Both calendar flags are known in advance and are kept as features; price is excluded from the models because future prices are not part of the specified feature set (a limitation revisited in Section X).

Heterogeneity of demand. Applying the Syntetos–Boylan–Croston classification [10], with pre-launch zeros removed, yields 21 intermittent, 9 lumpy, 9 smooth and 1 erratic SKU on the full history (Fig. 4). Recomputed on training data only, before the first origin, the counts are 22, 9, 8 and 1; the latter are used for all error analysis so that no future information enters the grouping. Most SKUs are slow and sporadic, which means one model may not suit all of them and results must be reported by class.

V. Feature Engineering

A. Feature Set

Table IV lists the twelve features used by the gradient-boosting model. Lagged sales and rolling means carry the SKU's recent level and weekly shape; calendar features carry effects known in advance; the integer item code, declared categorical, lets one pooled model learn SKU-specific levels. The global LSTM consumes the same information in sequence form: 56 days of scaled sales plus five calendar covariates (sine and cosine of the day of week, weekend, event and SNAP flags) for both the look-back and the 28 forecast days.

B. Leakage Safety

Let $\mathcal{F}_{t-1}$ denote all sales observed up to day $t-1$ together with the full calendar, which is known in advance. A feature

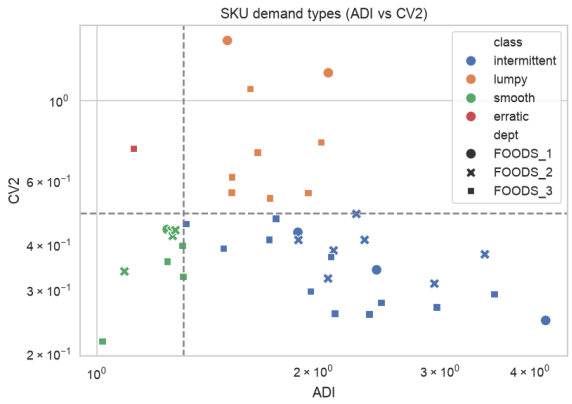


Fig. 4. Syntetos–Boylan classification of the 40 SKUs (ADI vs. CV^2; dashed lines at 1.32 and 0.49). Source: notebook 01, Sec. 13.

TABLE III
EDA findings and the design decisions they drove

Finding	Decision
ACF peaks at 7, 14, 21, 28	Lags 1/7/14/28; lag-7 naive baseline; weekly Kalman states
Moving level, occasional shifts	Kalman level and trend states
SNAP uplift 10%; events mixed	Calendar covariates, known in advance
Sparse counts, 42.9% zeros	Clip at 0; MAE, WAPE, pinball; no zero imputation
Heterogeneous ADI/CV^2	Global model with item embedding; per-class reporting
Proxy marks pre-launch only	Diagnostic flag; limitation reported

is *leakage-safe* if its value on day t is a function of $\mathcal{F}_{t-1}$ alone. Lags satisfy this by construction through `shift(k)`. Rolling means are computed on `shift(1)` before the window is applied, so the window for day t covers $t-w,\ldots,t-1$. Following Kaufman et al. [25], we test this property rather than assert it:

Feature test. All sales from day $t = 1500$ onward are increased by 1,000 and the feature table is rebuilt. Every lag and rolling feature for days up to t is unchanged, while features for day $t+1$ do change (confirming the test is sensitive).

End-to-end model test. At the final origin, every post-origin sale is replaced by 999 and each of the six models is rerun. All forecasts are identical to the original run (tolerance 10^{-4}), demonstrating that no model, including the recursive gradient-boosting forecaster and the neural network, reads the future.

VI. Methodology

A. Problem Formulation

Let $y_{i,t} \in \mathbb{N}_0$ be the unit sales of SKU $i \in \{1,\ldots,N\}$, $N = 40$, on day t, and let $\mathbf{c}_t$ be the calendar covariates. At a forecast origin τ, a forecaster receives $\mathcal{F}_\tau = \{y_{j,s} : s \le \tau\} \cup \{\mathbf{c}_s\}$ and must return, for every SKU and horizon $h \in \{1,\ldots,H\}$, $H = 28$, a point forecast $\hat{y}_{i,\tau+h}$ and, where supported, quantile forecasts $\hat{q}^{(\alpha)}_{i,\tau+h}$ for $\alpha \in \{0.1, 0.5, 0.9\}$. All forecasters share one interface, `model(wide, cal, end,`

TABLE IV
FEATURES FOR SKU i ON DAY t

Feature	Definition	Rationale
`lag_k`, $k \in \{1, 7, 14, 28\}$	$y_{i,t-k}$	ACF peaks; recent value
`roll_mean_w`, $w \in \{7, 28\}$	$\frac{1}{w}\sum_{j=1}^{w} y_{i,t-j}$	Recent level
`dayofweek`, `is_weekend`	Calendar	53% weekend uplift
`month`	Calendar	Mild annual pattern
`is_event`, `snap`	Calendar	Known in advance
`item_code`	Categorical id	SKU-specific level

Source: `src/features.py`.

`horizon)`, and may read only rows before `end`. Fig. 5 shows the pipeline.

B. Seasonal Naive Baselines

The seasonal-naive forecaster with period m repeats the last observed season:

$$\hat{y}_{i,\tau+h} = y_{i,\ \tau-m+1+((h-1) \bmod m)}. \tag{2}$$

With $m = 7$ it predicts "the same weekday last week"; $m = 28$ is a four-week variant. A model that cannot beat this benchmark adds no value [20].

C. Pooled Gradient Boosting

Gradient boosting [3] approximates the regression function by an additive expansion

$$F_M(\mathbf{x}) = F_0 + \nu \sum_{m=1}^{M} f_m(\mathbf{x}), \tag{3}$$

where each tree f_m is fitted to the negative gradient of the loss at F_{m-1}; under squared error these are the current residuals, and ν is the learning rate. We use scikit-learn's `HistGradientBoostingRegressor` [26], which bins features into histograms in the manner of LightGBM [11], with $M = 200$, $\nu = 0.1$, maximum depth 6, squared-error loss and `item_code` as a native categorical feature. A single model is trained on all SKU-days before the origin (about 72,000 rows at the first origin).

Because lag features beyond day $\tau+1$ are unknown, the 28-day forecast is produced *recursively* [24]: the model predicts day $\tau + 1$, the clipped prediction is written into the history, features for $\tau+2$ are rebuilt from it, and so on. This keeps the model leakage-free but lets errors compound with the horizon.

D. Local Kalman Filter

Local linear trend (LLT). For each SKU independently, the hidden state $\mathbf{x}_t = [\ell_t, b_t]^\top$ holds the level and slope:

$$\mathbf{x}_t = \mathbf{F}\mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{F} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}), \tag{4}$$

$$y_t = \mathbf{H}\mathbf{x}_t + v_t, \qquad \mathbf{H} = (1 \ \ 0), \qquad v_t \sim \mathcal{N}(0, R). \tag{5}$$

The filter alternates a prediction step,

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\hat{\mathbf{x}}_{t-1}, \qquad \mathbf{P}_{t|t-1} = \mathbf{F}\mathbf{P}_{t-1}\mathbf{F}^\top + \mathbf{Q}, \tag{6}$$

with an update step using the Kalman gain $\mathbf{K}_t$:

$$S_t = \mathbf{H}\mathbf{P}_{t|t-1}\mathbf{H}^\top + R, \qquad \mathbf{K}_t = \mathbf{P}_{t|t-1}\mathbf{H}^\top S_t^{-1}, \tag{7}$$

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t\,(y_t - \mathbf{H}\hat{\mathbf{x}}_{t|t-1}), \tag{8}$$

$$\mathbf{P}_t = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\,\mathbf{P}_{t|t-1}. \tag{9}$$

The gain weighs the new observation against the model prediction: a large ratio $\mathbf{Q}/R$ makes the filter react quickly, a small one makes it smooth.

Weekly seasonal extension. Six dummy-seasonal states are appended, with $s_t = -\sum_{j=1}^{6} s_{t-j} + \omega_t$, giving an 8-dimensional state and $\mathbf{H} = (1, 0, 1, 0, 0, 0, 0, 0)$ [2]. This lets the filter learn a day-of-week profile that the LLT model cannot represent.

Noise calibration. Rather than maximise the likelihood [7], we set the noise with a fixed heuristic. R is the variance of the last 365 training days around their weekday means, floored at 0.1; then $\mathbf{Q} = \mathrm{diag}(0.05R,\ 0.0005R)$ for LLT, with an additional $0.001R$ on the leading seasonal state for the weekly model. Initialisation uses ℓ_0 equal to the mean of the first seven days, zero slope and seasonals, and a diffuse $\mathbf{P}_0 = 100\mathbf{I}$.

Forecasts and intervals. The h-step forecast applies the prediction step h times without updates; the predictive variance $V_h = \mathbf{H}\mathbf{P}_{\tau+h|\tau}\mathbf{H}^\top + R$ grows with h. Assuming Gaussian errors,

$$\hat{q}_{\tau+h}^{(0.1/0.9)} = \max\!\big(0,\ \hat{y}_{\tau+h} \mp z_{0.9}\sqrt{V_h}\big), \quad z_{0.9} = 1.2816. \tag{10}$$

E. Global Quantile LSTM

Inputs and scaling. For SKU i and forecast start t, the network sees the previous $L = 56$ days. To make large and small SKUs comparable, sales are divided by $\kappa_{i,t} = 1 + \frac{1}{L}\sum_{j=1}^{L} y_{i,t-j}$, in the spirit of DeepAR's scaling [4]. Each input step is $\mathbf{u}_s = [\,y_{i,s}/\kappa_{i,t},\ \mathbf{c}_s\,] \in \mathbb{R}^6$, with $\mathbf{c}_s$ the five calendar covariates.

Recurrent encoder. An LSTM cell [12] with hidden size 32 processes the look-back:

$$\mathbf{i}_s = \sigma(\mathbf{W}_i\mathbf{u}_s + \mathbf{U}_i\mathbf{h}_{s-1} + \mathbf{b}_i), \tag{11}$$

$$\mathbf{f}_s = \sigma(\mathbf{W}_f\mathbf{u}_s + \mathbf{U}_f\mathbf{h}_{s-1} + \mathbf{b}_f), \tag{12}$$

$$\mathbf{o}_s = \sigma(\mathbf{W}_o\mathbf{u}_s + \mathbf{U}_o\mathbf{h}_{s-1} + \mathbf{b}_o), \tag{13}$$

$$\tilde{\mathbf{c}}_s = \tanh(\mathbf{W}_c\mathbf{u}_s + \mathbf{U}_c\mathbf{h}_{s-1} + \mathbf{b}_c), \tag{14}$$

$$\mathbf{m}_s = \mathbf{f}_s \odot \mathbf{m}_{s-1} + \mathbf{i}_s \odot \tilde{\mathbf{c}}_s, \qquad \mathbf{h}_s = \mathbf{o}_s \odot \tanh(\mathbf{m}_s). \tag{15}$$

The forget gate $\mathbf{f}_s$ lets the cell memory $\mathbf{m}_s$ retain weekly structure across the 56-day window without the vanishing gradients of a plain RNN.

Decoder head. The final hidden state is concatenated with a 4-dimensional learned item embedding $\mathbf{e}_i$ and the flattened calendar covariates of the 28 forecast days, $\mathbf{z} = [\mathbf{h}_L; \mathbf{e}_i; \mathrm{vec}(\mathbf{C}_{t:t+27})] \in \mathbb{R}^{176}$, and mapped by a two-layer perceptron ($176 \to 64 \to 84$, ReLU) to a 28×3 matrix of

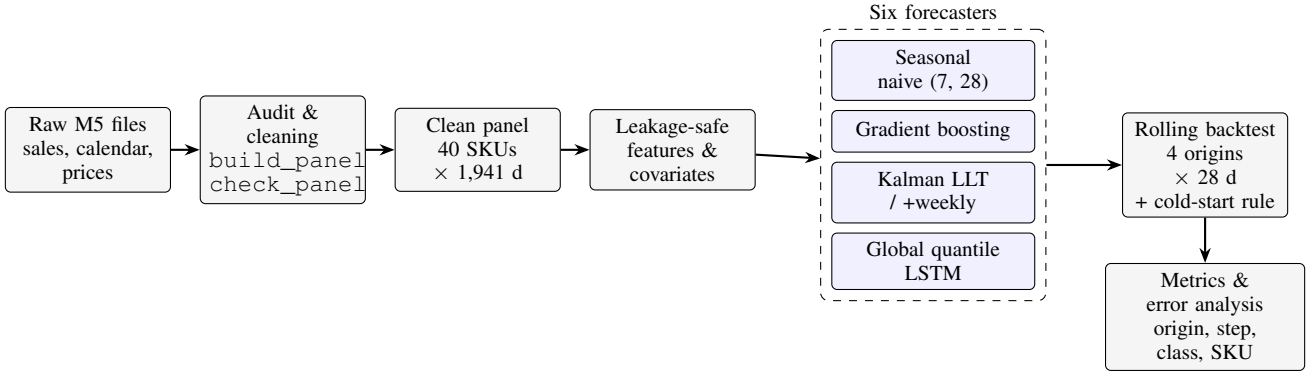


Fig. 5. End-to-end pipeline. Every forecaster plugs into the same backtest loop, which guarantees identical SKUs, origins and horizons.

scaled quantiles. The model has 22,068 trainable parameters. Forecasts for all horizons are emitted at once (direct multi-horizon, as in MQ-RNN [13]), so no prediction is fed back as input.

Loss. Training minimises the pinball loss [15] averaged over quantiles and horizons,

$$\mathcal{L} = \frac{1}{3H} \sum_{\alpha} \sum_{h=1}^{H} \rho_\alpha \Big(\tilde{y}_{t+h-1} - \hat{\tilde{q}}^{(\alpha)}_{t+h-1} \Big), \qquad (16)$$

with $\rho_\alpha(u) = \max(\alpha u, (\alpha - 1)u)$, where tildes denote scaled values. The minimiser of $\mathbb{E}[\rho_\alpha(Y - q)]$ is the α-quantile of Y; with $\alpha = 0.9$, under-forecasting is penalised nine times more than over-forecasting. Because quantiles are equivariant under positive scaling, outputs are multiplied back by $\kappa_{i,t}$. Any crossing of the three outputs is removed by sorting (monotone rearrangement [17]), and results are clipped at zero. The P50 output serves as the point forecast.

Training. At each origin the network is trained from scratch on windows whose starts lie in $[L, \tau - 2H]$ (about 68,700 SKU-windows at the first origin), using Adam [27] with learning rate 10^{-3}, batch size 256 and at most 20 epochs. The 28 most recent window starts form the validation set for early stopping with patience 3, and the best weights are restored. The implementation uses PyTorch [28].

F. Cold-Start Fallback

A SKU with fewer than 56 days since its first sale at the origin receives, for each forecast day, the median daily sales of the established SKUs on the same weekday over the last 28 days; its intervals are left undefined. None of the 40 SKUs triggers the rule, so it was tested by erasing all but the last ten days of one SKU's history and confirming that every model returns the fallback for that SKU while other SKUs are unaffected.

VII. Experimental Design

A. Rolling-Origin Backtest

Four origins, 28 days apart, cover the last 112 days of data (Table V). At each origin every model is refitted on data before the origin and forecasts the next 28 days for all 40 SKUs.

TABLE V
Forecast origins

#	Train until	Train days	Forecast from	Forecast to
1	2016-01-31	1,829	2016-02-01	2016-02-28
2	2016-02-28	1,857	2016-02-29	2016-03-27
3	2016-03-27	1,885	2016-03-28	2016-04-24
4	2016-04-24	1,913	2016-04-25	2016-05-22

Source: notebook 02, Sec. 3.

Algorithm 1 Rolling-origin backtest

Require: sales matrix $\mathbf{Y}$, calendar $\mathbf{C}$, models $\mathcal{M}$, origins $\mathcal{T}$, horizon H
for each origin $\tau \in \mathcal{T}$ **do**
  $\mathbf{A} \leftarrow \mathbf{Y}_{\tau:\tau+H}$ ▷ actuals, held out
  $\mathcal{C} \leftarrow \{i : \text{history}_i(\tau) < 56\}$ ▷ cold-start SKUs
  for each model $m \in \mathcal{M}$ **do**
    $(\hat{\mathbf{Y}}, \hat{\mathbf{Q}}_{0.1}, \hat{\mathbf{Q}}_{0.9}) \leftarrow m(\mathbf{Y}_{:\tau}, \mathbf{C}, H)$
    replace columns $i \in \mathcal{C}$ by the fallback forecast
    append rows (origin, date, step, SKU, m, actual, forecasts)
  end for
end for
return tidy prediction table

This yields $4 \times 28 \times 40 = 4{,}480$ scored SKU-days per model, containing 13,961 units, 40.3% of them zero days. Algorithm 1 states the procedure. A fairness assertion confirms that all six models are scored on exactly the same {origin, date, SKU, actual} rows.

B. Evaluation Metrics

For n scored SKU-days with actuals y_k, point forecasts $\hat{y}_k$ and quantile forecasts $\hat{q}^{(\alpha)}_k$:

$$\text{MAE} = \tfrac{1}{n} \textstyle\sum_k |y_k - \hat{y}_k|, \qquad (17)$$

$$\text{RMSE} = \sqrt{\tfrac{1}{n} \textstyle\sum_k (y_k - \hat{y}_k)^2}, \qquad (18)$$

$$\text{WAPE} = \textstyle\sum_k |y_k - \hat{y}_k| \,/\, \sum_k y_k, \qquad (19)$$

$$\text{PL}_\alpha = \tfrac{1}{n} \textstyle\sum_k \rho_\alpha \big(y_k - \hat{q}^{(\alpha)}_k \big), \qquad (20)$$

$$\text{Cov} = \tfrac{1}{n} \textstyle\sum_k \mathbb{I}[\hat{q}^{(0.1)}_k \le y_k \le \hat{q}^{(0.9)}_k]. \qquad (21)$$

WAPE is a volume-weighted scale-free error that, unlike MAPE, remains defined when individual actuals are zero [22]; it is returned as undefined when all actuals in a group are

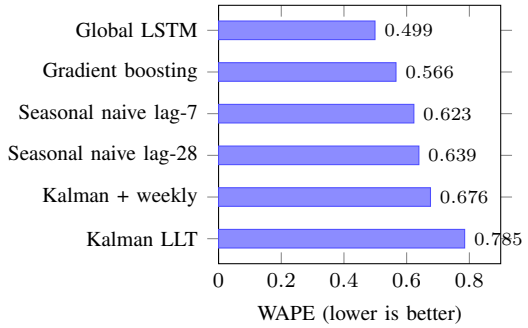


Fig. 6. Pooled WAPE by model. Source: `outputs/metrics.csv`.

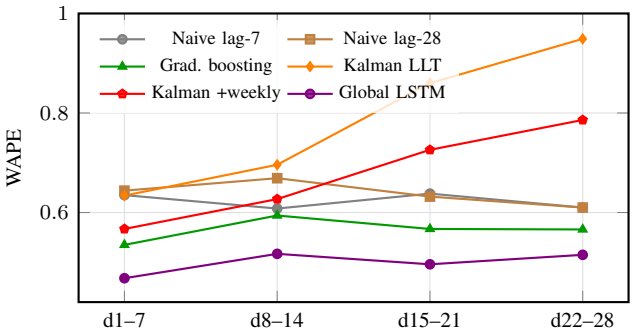


Fig. 7. WAPE by weekly horizon block, pooled over origins and SKUs. Source: computed from `outputs/predictions.csv`.

zero. Ideal coverage of the P10–P90 interval is 80%; we also report mean interval width, because a wide interval can achieve coverage while being useless for stock decisions. Point metrics use P50 for probabilistic models.

C. Implementation and Reproducibility

The code is organised as a Python package (`src/`: `data`, `features`, `baselines`, `kalman`, `global_model`, `backtest`, `metrics`, `plots`) driven by two notebooks, with the clean panel committed to the repository so that the modelling notebook runs without the raw files. Metric functions are checked against hand-computed values, and the Kalman filter against three synthetic series with known answers (constant level, linear trend recovering slope 0.484 versus a true 0.5, and a weekly pattern). The complete six-model backtest runs in 239 s on a laptop CPU. The repository is public at https://github.com/SayAn1-dls/AI_DL_SEM5.

VIII. Results

A. Overall Accuracy

Table VI reports all metrics pooled over 40 SKUs, four origins and 28 days, and Fig. 6 visualises WAPE. The global LSTM is best on every point metric (MAE 1.555, RMSE 3.533, WAPE 0.499). Gradient boosting is second (WAPE 0.566) and is the only other model to beat the lag-7 seasonal-naive baseline (0.623). Both Kalman variants lose to that baseline; adding weekly seasonality improves the filter's WAPE from 0.785 to 0.676, confirming that the weekly cycle identified in Section IV is the dominant structure.

B. Probabilistic Forecasts

The LSTM's P10–P90 interval covers 81.2% of outcomes, within 1.2 points of the nominal 80%, with a mean width of 5.98 units. The Kalman intervals cover 94.5% (LLT) and 95.0% (weekly) with mean widths of 11.11 and 10.60 units. They are valid but over-dispersed: they achieve coverage by being almost twice as wide, which in an inventory setting translates into excess safety stock. The pinball loss confirms the ranking at every quantile level, with the largest gap at P50 (0.778 versus 1.054) and P90 (0.489 versus 0.837). Calibration of the LSTM is consistent across demand classes (coverage 0.778 lumpy to 0.866 erratic), whereas the Kalman filters overcover in every class (0.938 to 0.954).

C. Stability Across Origins

Table VII shows WAPE at each origin. The LSTM is best at all four origins, varying only between 0.478 and 0.520. Gradient boosting beats lag-7 seasonal naive at three of four origins and is narrowly worse at the first (0.630 versus 0.583). The Kalman filters degrade sharply at origin 3 (LLT 1.089, weekly 0.946); Section VIII-H traces this to a single SKU.

D. Error by Forecast Horizon

Fig. 7 groups errors into weekly blocks. The weekly Kalman filter is competitive only in the first week, where its WAPE (0.567) beats both seasonal-naive variants (0.635 and 0.644); it then deteriorates steadily to 0.786 in days 22–28, and the LLT filter reaches 0.949. Gradient boosting (0.535 to 0.594) and the LSTM (0.468 to 0.517) are nearly flat. The divergence reflects the two models' information sets: the filter extrapolates its last level and slope and knows nothing about the future calendar, whereas the LSTM conditions directly on the calendar of every forecast day.

E. Error by Demand Class

Table VIII disaggregates WAPE and MAE by the training-period demand class. The LSTM is best in all four classes on both metrics. Lumpy SKUs are the hardest for every model (WAPE 0.839 to 1.143). The low WAPE of the intermittent class requires care in interpretation: that class contains FOODS_3_090, the highest-volume SKU in the panel, which is classified as intermittent because of extended zero periods in its history and accounts for 65% of the class's test volume.

F. Per-SKU Results and Volume Concentration

Pooled metrics are dominated by high-volume SKUs, and the concentration here is extreme: FOODS_3_090 alone supplies 5,992 of the 13,961 test units (42.9%). Two checks guard against the conclusion resting on that one series. First, at the level of individual SKUs, the LSTM has the lowest MAE on 31 of 40 SKUs, gradient boosting on 5, the weekly Kalman filter on 3 and the LLT filter on 1, and the LSTM also has the lowest median per-SKU WAPE (0.912, against 0.936 for gradient boosting and 0.979 for the weekly Kalman filter; Fig. 8). Second, after removing FOODS_3_090 entirely, the

TABLE VI
OVERALL RESULTS: 40 SKUS, 4 ORIGINS, 28-DAY HORIZON (4,480 SKU-DAYS PER MODEL). LOWER IS BETTER EXCEPT COVERAGE (IDEAL 0.80). BEST VALUES IN BOLD.

Model	MAE	RMSE	WAPE	$PL_{0.1}$	$PL_{0.5}$	$PL_{0.9}$	Coverage P10–P90	Mean width
Seasonal naive (lag-7)	1.940	4.310	0.623	–	–	–	–	–
Seasonal naive (lag-28)	1.992	4.511	0.639	–	–	–	–	–
Gradient boosting	1.763	3.672	0.566	–	–	–	–	–
Kalman (level+trend)	2.445	6.475	0.785	0.307	1.223	0.890	0.945	11.11
Kalman (+weekly)	2.108	5.025	0.676	0.304	1.054	0.837	0.950	10.60
Global LSTM (quantile)	**1.555**	**3.533**	**0.499**	**0.290**	**0.778**	**0.489**	**0.812**	**5.98**

Source: `outputs/metrics.csv`*; interval widths computed from* `outputs/predictions.csv`.

TABLE VII
WAPE BY FORECAST ORIGIN (LAST TRAINING DAY)

Model	01-31	02-28	03-27	04-24
Seasonal naive (lag-7)	0.583	0.557	0.707	0.644
Seasonal naive (lag-28)	0.664	0.611	0.659	0.625
Gradient boosting	0.630	0.529	0.627	0.490
Kalman (level+trend)	0.696	0.687	1.089	0.685
Kalman (+weekly)	0.619	0.578	0.946	0.581
Global LSTM	**0.520**	**0.481**	**0.520**	**0.478**

Source: computed from `outputs/predictions.csv`.

TABLE VIII
WAPE (AND MAE) BY SYNTETOS–BOYLAN CLASS, TRAINING-PERIOD CLASSIFICATION

Model	Smooth (8)	Erratic (1)	Interm. (22)	Lumpy (9)
Naive lag-7	0.759 (2.01)	0.710 (3.76)	0.493 (1.85)	1.088 (1.91)
Naive lag-28	0.775 (2.05)	0.639 (3.38)	0.508 (1.90)	1.142 (2.00)
Grad. boosting	0.592 (1.57)	0.552 (2.92)	0.475 (1.78)	1.008 (1.77)
Kalman LLT	0.797 (2.11)	0.663 (3.51)	0.721 (2.70)	1.143 (2.00)
Kalman +weekly	0.736 (1.95)	0.686 (3.63)	0.578 (2.17)	1.106 (1.94)
Global LSTM	**0.574 (1.52)**	**0.540 (2.86)**	**0.412 (1.54)**	**0.839 (1.47)**
Test-volume share	17.0%	4.2%	66.1%	12.6%

Source: computed from `outputs/predictions.csv`*; classes from data before 2016-02-01.*

ranking is unchanged: LSTM 0.677, gradient boosting 0.789, seasonal naive 0.879, weekly Kalman 0.906 and LLT Kalman 0.976. Absolute WAPE rises for every model without that SKU because the remaining series are slower and noisier, but the order of the models does not change.

One detail deserves mention. Averaged over SKUs, gradient boosting's per-SKU WAPE (mean 1.204) is worse than that of seasonal naive (1.131), even though its median is better. A few slow SKUs receive large relative errors from the pooled tree model, a heavy tail that the pooled metric conceals.

G. Seed Stability

Neural network training depends on random initialisation. Repeating the full four-origin LSTM backtest with seeds 0, 1 and 2 gives WAPE 0.501 ± 0.003, MAE 1.560 ± 0.011, RMSE 3.577 ± 0.074 and coverage 0.816 ± 0.003 (mean $\pm$ standard deviation). The worst seed (WAPE 0.504) still beats gradient boosting by 0.062, so the LSTM's lead is not an artefact of a favourable initialisation.

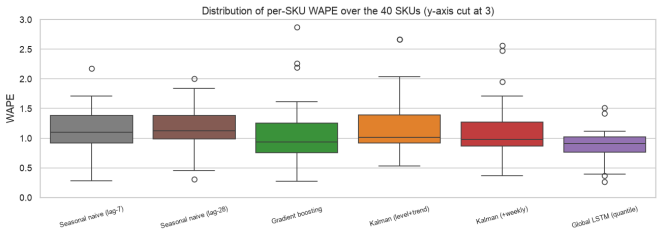


Fig. 8. Distribution of per-SKU WAPE across the 40 SKUs (axis cut at 3). Source: notebook 02, Sec. 7.4.

H. Case Study: Trend Extrapolation in the Kalman Filter

At origin 3 the LLT filter forecast 2,878 units of FOODS_3_090 over 28 days against 1,423 actual, an over-forecast of 102%; at the other origins its 28-day totals were 1,441 versus 1,404, 2,095 versus 1,513 and 2,002 versus 1,652. A short run of high sales before the origin pushed the slope state upward, and because the forecast step multiplies by $\mathbf{F}$ repeatedly, a positive slope compounds linearly over 28 days. This single failure explains most of the filter's spike at origin 3 in Table VII, and it illustrates the structural weakness of an unconstrained trend for retail demand, which rarely sustains a linear drift over a month.

Fig. 9 compares the two families' intervals on the same SKUs and window: the Kalman band widens with the horizon regardless of the data, whereas the LSTM band follows the weekly pattern.

IX. DISCUSSION

A. Answers to the Research Questions

RQ1: Global versus local. The global model is more accurate, and the result is robust: it wins at every origin, in every demand class, in every horizon block, on 31 of 40 SKUs, across three seeds and after removing the dominant SKU. This agrees with the M5 finding that pooled learners outperformed local statistical methods [5] and with the argument of Montero-Manso and Hyndman that a global model can share structure across series without sacrificing fit [6]. The mechanism is visible in the data: most SKUs are intermittent or lumpy, and a single intermittent series contains too few non-zero observations to estimate a weekly profile reliably, whereas a model pooled over 40 series can.

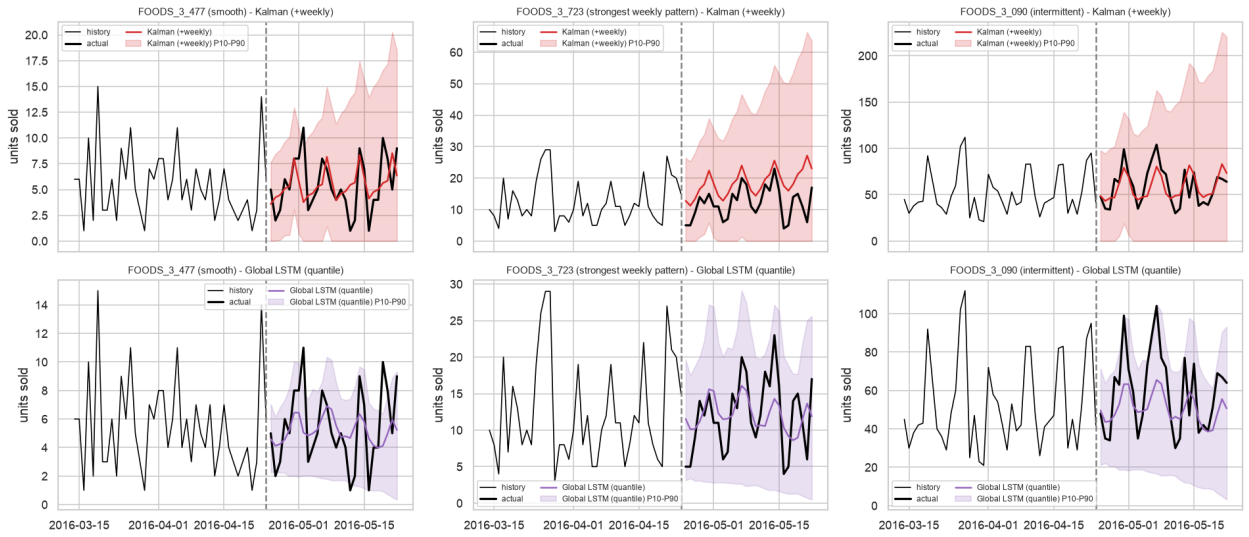


Fig. 9. P10–P90 intervals for three representative SKUs at the final origin: weekly Kalman filter (top) versus global LSTM (bottom). Black lines are actuals. Source: notebook 02, Sec. 9.1.

RQ2: When is the local filter competitive? Rarely, and only briefly. In the first forecast week the weekly Kalman filter beats seasonal naive (0.567 versus 0.635), consistent with its ability to track the current level. It never beats gradient boosting or the LSTM in any week, class or origin, and its advantage over naive disappears by the second week. We therefore do not find a regime in this data where the local filter is the best choice.

RQ3: Calibration. The LSTM's quantiles are close to nominal and sharp. The Kalman intervals are conservative for two identifiable reasons: the Gaussian predictive variance grows without bound under the LLT dynamics, and the heuristic noise setting was never fitted to the data. A filter with likelihood-estimated noise would likely be sharper, which is a concrete experiment for the next phase.

B. Why Not the Alternatives

Several credible alternatives were not adopted, and each loses for a specific reason in this setting. *DeepAR proper* [4], with a negative-binomial likelihood, is better matched to count data than our direct quantile head, but its tooling (GluonTS) carried integration risk for Phase 1; the project specification explicitly allowed the quantile LSTM as a fallback, and DeepAR is the natural Phase 2 upgrade. *Croston-type methods* [8], [9] are purpose-built for intermittent demand and should have been included as a baseline, since 22 of the 40 SKUs are intermittent; their omission is a genuine gap rather than a reasoned rejection. *A more heavily tuned gradient-boosting model* with a Tweedie or Poisson objective, close to winning M5 practice [5], could narrow the gap to the LSTM; our model used one fixed configuration and squared-error loss, so the 0.067 WAPE gap should be read as an upper bound on the LSTM's advantage over well-engineered tabular learning. *Transformer models* such as TFT [14] are unlikely to pay for their complexity on 40 series. Gradient boosting remains the most attractive alternative to the LSTM: it is second on every pooled metric, trains in seconds, and is easy to explain, so a team without deep-learning infrastructure would accept a WAPE about 13% higher (0.566 versus 0.499) by choosing it.

C. Threats to Validity

Construct validity. The LSTM's point forecast is a median, while the other models forecast means. The median minimises absolute error [23], so MAE and WAPE favour it on zero-heavy counts. The LSTM also wins on RMSE, which favours means, but by a smaller margin (3.533 versus 3.672), so part of its MAE advantage reflects the choice of functional rather than skill. Pooled metrics are dominated by one SKU, which we addressed with the per-SKU and exclusion analyses above.

Internal validity. The LSTM's early-stopping validation windows are the 28 most recent window positions before the origin; their target days overlap the latest training targets by up to 27 days, so the validation loss is somewhat optimistic. This affects only the choice of stopping epoch and never exposes test data, which the end-to-end leakage test confirms. All models use a single, untuned configuration, and the Kalman noise is heuristic, so the comparison is of reasonable defaults rather than of each method at its best. No formal significance test (e.g. Diebold–Mariano [29]) has yet been applied.

External validity. Results come from one store, one category, 40 SKUs and four consecutive windows in February–May 2016. They may not transfer to other stores, categories, seasons or the full 30,490-series hierarchy. The data come from a public mirror of the official files; shapes match, but byte-level identity was not verified.

X. Limitations and Future Work

The limitations above translate directly into a sequenced plan. Each item states a measurable test of completion.

Strengthen the classical learner. Tune gradient boosting with nested rolling-origin validation and compare squared-error, Poisson and Tweedie objectives, adding LightGBM [11]. *Done when* a tuned model's WAPE and per-SKU median WAPE are reported against the current 0.566 and 0.936.

Add an intermittent-demand baseline. Implement Croston and its SBA correction [8], [9]. *Done when* its WAPE on the 22 intermittent SKUs is reported alongside Table VIII.

Estimate the Kalman noise by maximum likelihood [7] and consider a damped trend. *Done when* coverage is closer to 0.80 than the current 0.950 with mean width below 10.6 units.

Richer features with ablation. Add price, price change, rolling standard deviation and days-since-launch features, and remove feature groups one at a time. *Done when* an ablation table quantifies each group's WAPE contribution and feature importances are reported.

Statistical testing and scale-free metrics. Apply Diebold–Mariano tests [29] to the LSTM versus gradient-boosting loss differentials, and add the M5 RMSSE metric [5]. *Done when* p-values and RMSSE appear in the results table.

Phase 2: deep learning. Fix the validation split so that validation targets are disjoint from training targets; replace the quantile head with a DeepAR-style negative-binomial likelihood [4]; ablate the item embedding, the look-back length (28, 56, 112 days) and the future covariates; and add explicit regularisation (dropout, weight decay) beyond early stopping. A hybrid candidate with a clear rationale is to supply the Kalman filter's filtered level, slope and variance as additional inputs to the global model, combining the filter's fast level tracking with the network's calendar awareness. Because the LSTM already outperforms the filter even in the first week, such a hybrid must be justified by an ablation showing a measurable gain over the plain LSTM, not assumed to help.

Scale. Rerun the cheaper models on all 1,437 CA_1 FOODS series. *Done when* the model ranking is reported at that scale.

XI. Conclusion

Under an identical, leakage-tested rolling backtest on 40 M5 FOODS SKUs, a single global quantile LSTM forecast daily demand more accurately than per-SKU Kalman filters, pooled gradient boosting and seasonal-naive baselines, and produced near-nominal 80% intervals at about half the width of the Kalman intervals. The advantage held across origins, horizons, demand classes, seeds and the removal of the dominant SKU. The local Kalman filter was competitive only against naive forecasts and only in the first week; its trend extrapolation and conservative noise setting were its main failures. Gradient boosting proved a strong and inexpensive second choice. For a retailer forecasting many related, mostly intermittent items, these results favour global models that share information across SKUs and condition on the known future calendar, with gradient boosting as the practical default where deep-learning infrastructure is unavailable.

Appendix A
Repository Structure and Reproduction

The repository has five top-level parts. The first notebook, `01_data_cleaning_eda`, performs the audit, cleaning, proxy construction and EDA in about one minute. The second, `02_model_comparison`, builds features, runs the leakage tests, models, backtest and error analysis in about 15 minutes including the seed study. The `src/` package holds the modules described in Section VII; `data/processed/` holds the clean panel; and `outputs/` holds all predictions, metric tables and figures. Reproduction requires installing `requirements.txt`, placing the three M5 files in `data/raw/` (needed only by the first notebook) and executing the notebooks in order. The second notebook ends with an automated acceptance check of eleven criteria, all of which pass.

Appendix B
Individual Contributions

Member	Primary responsibilities
Sayan Bhattacharya	*Cleaning Pipeline*
Aryu Rao	*Demand Classification, Data Audit*
Debasish Karan	*Feature Engineering*
Saswataduity Bhuin	*EDA*
Aryan Vibhuti	*Model Training*

Acknowledgment

The authors thank the course faculty for guidance, and the M5 organisers and Walmart for releasing the data.

References

[1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[2] A. C. Harvey, *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge, U.K.: Cambridge University Press, 1989.

[3] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[4] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *International Journal of Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.

[5] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.

[6] P. Montero-Manso and R. J. Hyndman, "Principles and algorithms for forecasting groups of time series: Locality and globality," *International Journal of Forecasting*, vol. 37, no. 4, pp. 1632–1653, 2021.

[7] J. Durbin and S. J. Koopman, *Time Series Analysis by State Space Methods*, 2nd ed. Oxford, U.K.: Oxford University Press, 2012.

[8] J. D. Croston, "Forecasting and stock control for intermittent demands," *Operational Research Quarterly*, vol. 23, no. 3, pp. 289–303, 1972.

[9] A. A. Syntetos and J. E. Boylan, "The accuracy of intermittent demand estimates," *International Journal of Forecasting*, vol. 21, no. 2, pp. 303–314, 2005.

[10] A. A. Syntetos, J. E. Boylan, and J. D. Croston, "On the categorization of demand patterns," *Journal of the Operational Research Society*, vol. 56, no. 5, pp. 495–503, 2005.

[11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017.

[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] R. Wen, K. Torkkola, B. Narayanaswamy, and D. Madeka, "A multi-horizon quantile recurrent forecaster," arXiv preprint arXiv:1711.11053, 2017.

[14] B. Lim, S. Ö. Arık, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.

[15] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.

[16] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.

[17] V. Chernozhukov, I. Fernández-Val, and A. Galichon, "Quantile and probability curves without crossing," *Econometrica*, vol. 78, no. 3, pp. 1093–1125, 2010.

[18] S. Kolassa, "Evaluating predictive count data distributions in retail sales forecasting," *International Journal of Forecasting*, vol. 32, no. 3, pp. 788–803, 2016.

[19] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: An analysis and review," *International Journal of Forecasting*, vol. 16, no. 4, pp. 437–450, 2000.

[20] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[21] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[22] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.

[23] S. Kolassa, "Why the "best" point forecast depends on the error or accuracy measure," *International Journal of Forecasting*, vol. 36, no. 1, pp. 208–211, 2020.

[24] S. Ben Taieb, G. Bontempi, A. F. Atiya, and A. Sorjamaa, "A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition," *Expert Systems with Applications*, vol. 39, no. 8, pp. 7067–7083, 2012.

[25] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[26] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd International Conference on Learning Representations (ICLR)*, 2015.

[28] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019.

[29] F. X. Diebold and R. S. Mariano, "Comparing predictive accuracy," *Journal of Business & Economic Statistics*, vol. 13, no. 3, pp. 253–263, 1995.